Tenarai AppSec Engineer — Interview Prep

Oct 5, 2026 · @Nanda

Company context and how to position yourself

Tenarai is Infogain rebranded as an enterprise AI services company, so this is a consulting/delivery role: you will secure client workloads on AWS and explain risk to client security teams, not own one product. (tenarai.com)

What to know about them

- Formerly Infogain; HQ in Los Gatos, Silicon Valley; 5,500+ employees; clients are Fortune 500 in tech, retail/CPG, travel, insurance, healthcare and telecom.
- Partners include AWS, Google Cloud, Microsoft, Databricks, Salesforce, Adobe and Anthropic.
- They sell agentic AI: "Hybrid Agentic PODs" (AI agents across the SDLC) and **Tenarai-IGNIS**, a secure AI access platform with guardrails, access controls, RAG and agent workflows.
- Implication: expect at least one question on securing LLM/agent apps (prompt injection, tool permissions, data leakage through RAG). This is where your AI-assisted security work becomes a differentiator.

Your fit vs. the JD (be honest, frame the gaps)

JD asks	Your evidence	How to frame it
8+ yrs AppSec	5+ yrs, 60+ assessments, 120+ high/critical findings	Density of hands-on work; offensive depth makes you a better reviewer and triager
3+ yrs AWS security	Cloud security in AppSec/pentest scope	Name specific AWS engagements; do not overstate. Prepare 2 concrete AWS stories
EKS/containers	KubeArmor + Cilium runtime security at AccuKnox	Strong card: runtime policy, eBPF, network policy, least privilege for workloads

JD asks	Your evidence	How to frame it
OWASP, API, auth	Web/API/mobile pentesting; 2 CVEs in Microweber	You find what SAST misses; you can write the remediation
AWS Security Specialty	CEH, CAP, CC	State if you are preparing for it and a target date
DevSecOps tooling	Burp daily; SAST/SCA exposure	Know Semgrep, Checkov, Trivy hands-on before the interview

60-second intro (adapt, say it out loud 5 times)

> I'm an application security engineer with 5+ years across web, API, mobile and cloud. I've done 60+ assessments and reported 120+ high and critical findings, plus two CVEs in Microweber CMS. At AccuKnox I worked on Kubernetes runtime security with KubeArmor and Cilium, so I'm comfortable securing containerized workloads at the runtime layer, not just in code. Recently I've focused on using AI in security workflows, which fits how Tenarai delivers with agentic engineering. I want to move from finding issues to building the guardrails that stop them shipping, across the SDLC on AWS.

If they raise the 8-year requirement: "I'm under 8 years on paper. What I'd ask you to weigh is the number of real findings and environments I've worked in, and I'm happy to go deep on any area in the JD right now."

Threat modeling and secure design review

Lead every answer with a process (scope, data flow, trust boundaries, threats, controls, owners) and close with how you track it to done.

Q1. Walk me through how you threat model a new microservice on AWS.

1. Scope and assets: what data it handles (PII, card data, tokens), who calls it, what it calls.
2. Draw a data flow diagram: clients → CloudFront/WAF → API Gateway or ALB → ECS/EKS/Lambda → RDS/DynamoDB/S3, plus queues (SQS/SNS) and third-party APIs. Mark trust boundaries (internet edge, VPC, account boundary, IAM role boundary).
3. Apply STRIDE per element and per flow crossing a boundary.
4. Rate with a simple likelihood × impact or DREAD-lite; agree risk owners.

5. Map each threat to a control and a ticket, and a test (unit test, DAST check, Config rule) so it stays fixed.
6. Revisit on major change: new data type, new external integration, new auth flow.

Q2. STRIDE applied to API Gateway + Lambda + DynamoDB?

- Spoofing: missing or weak authorizer; fix with Cognito/OIDC JWT authorizer, validate `aud`, `iss`, `exp`.
- Tampering: mass assignment into DynamoDB items; fix with schema validation (request validators, allow-listed fields).
- Repudiation: no audit trail; enable API Gateway access logs, CloudTrail data events, structured logs with user ID and request ID.
- Information disclosure: verbose errors, over-broad scan/query returning other tenants' items; fix with partition-key tenant scoping and IAM `dynamodb:LeadingKeys` conditions.
- Denial of service: no throttling, Lambda concurrency exhaustion; fix with usage plans, WAF rate rules, reserved concurrency.
- Elevation of privilege: Lambda role with `dynamodb:*` on `*`; fix with resource-scoped least privilege.

Q3. STRIDE vs PASTA — when would you use each?

STRIDE is fast and developer-friendly for per-feature design reviews. PASTA is a 7-stage, risk-and-business-driven process (objectives, technical scope, decomposition, threat analysis, vulnerability analysis, attack modeling, risk/impact). Use PASTA for a high-value system where the client wants business-impact language for executives; use STRIDE inside sprints.

Q4. How do you scale threat modeling across many delivery teams?

You can't attend every design meeting. Use a tiering questionnaire (internet-facing? handles PII/payments? new auth?) so only high-risk changes get a full review. Give teams templates and a library of pre-modeled patterns ("standard public API on API Gateway + Lambda"). Train security champions to run lightweight STRIDE sessions. Track threat model coverage as a metric.

Q5. What's the most common design flaw you find in microservices?

Broken object-level and function-level authorization between services: the edge authenticates the user, but internal services trust any request from inside the VPC. Answer: propagate identity (signed JWT or mTLS with service identity), authorize at each

service, never treat "internal" as trusted. Tie in a real IDOR/BOLA finding from your pentests here.

Q6. How would you threat model an LLM agent feature (relevant to Tenarai-IGNIS-type work)?

Add assets: system prompt, tools/actions, retrieved documents, conversation memory. Key threats: direct and indirect prompt injection (malicious content in RAG docs or web pages), excessive agency (tools with write access), sensitive data leaking across users through shared vector stores, insecure output handling (model output rendered as HTML or passed to a shell). Controls: least-privilege tool scopes, human approval for destructive actions, per-tenant retrieval filters, output encoding, logging prompts and tool calls. Reference the OWASP Top 10 for LLM Applications.

AWS security services and IAM

This is the section most likely to expose a gap; know each service's job in one sentence and one real configuration detail.

Service	One-line job	Detail worth saying
IAM	Who can do what on which resource	Policy evaluation: explicit Deny > SCP/RCP > permission boundary > session policy > identity/resource Allow
WAF	L7 filtering at CloudFront/ALB/API Gateway	Managed rule groups (Core, Known Bad Inputs, SQLi), rate-based rules, start in Count mode
Shield	DDoS protection	Standard is free L3/L4; Advanced adds DRT support and cost protection
GuardDuty	Threat detection from CloudTrail, VPC Flow, DNS logs	Plus EKS audit logs, S3, Lambda, runtime monitoring; e.g. finding for EC2 instance creds used outside AWS
Inspector	Vulnerability scanning	EC2, ECR images, Lambda functions and code; uses SSM agent, no separate scanner

Service	One-line job	Detail worth saying
Security Hub	Aggregates findings, runs standards	AWS FSBP, CIS benchmarks, PCI DSS; cross-account via delegated admin
KMS	Key management	Key policies are the primary control; CMKs give audit and rotation control; envelope encryption
Secrets Manager	Store and rotate secrets	Rotation via Lambda; resource policy; vs SSM Parameter Store (cheaper, no native rotation)
CloudTrail	API audit log	Org trail to a locked log-archive account, log file validation, data events for S3/Lambda

Q1. How do you design least-privilege IAM for an application?

One role per workload (ECS task role, Lambda execution role, IRSA/Pod Identity for EKS), never shared. Start from actions the code actually calls; use IAM Access Analyzer policy generation from CloudTrail activity to tighten. Scope resources by ARN, add conditions (`aws:SourceVpce` , `aws:ResourceTag` , `kms:ViaService`). Permission boundaries let dev teams create roles without escalating. SCPs at the org level block dangerous actions outright (leaving regions, disabling CloudTrail/GuardDuty). Review unused permissions with Access Analyzer.

Q2. Explain IAM policy evaluation for a cross-account request.

The request is allowed only if both sides allow it: the caller's identity policy in account A and the resource policy (or role trust policy) in account B. Any explicit Deny anywhere wins. SCPs in both orgs must also permit it. Mention the confused-deputy problem and `sts:ExternalId` for third-party role assumption.

Q3. How do you find IAM privilege escalation paths?

Look for permissions like `iam:PassRole` combined with `lambda:CreateFunction` or `ec2:RunInstances` , `iam:CreatePolicyVersion` , `iam:AttachRolePolicy` , `iam:UpdateAssumeRolePolicy` , `sts:AssumeRole` on wildcards. Tools: PMapper, Cloudsplaining, Prowler, Pacu (in authorized tests). Fix by scoping `PassRole` to specific role ARNs and `iam:PassedToService` .

Q4. An S3 bucket is public with customer data. Walk me through the response.

Contain first: enable Block Public Access at bucket and account level. Preserve evidence: S3 server access logs, CloudTrail data events for GetObject, to determine whether data was actually read and by whom. Scope: which objects, which IPs, time window. Notify per client process and regulation. Prevent recurrence: account-level Block Public Access via SCP, Config rule `s3-bucket-public-read-prohibited`, Checkov in the pipeline.

Q5. KMS: AWS-managed key vs customer-managed key?

Customer-managed keys let you write the key policy, restrict use to specific roles and services, disable or schedule deletion, enable rotation, and share cross-account. AWS-managed keys are simpler but you cannot control the policy. For regulated data (PCI, HIPAA) use CMKs with separation of duties: key administrators can't use the key, key users can't administer it.

Q6. How do you handle secrets in an application?

No secrets in code, env files in repos, container images, or Lambda env vars in plain text. Store in Secrets Manager, fetch at runtime with the workload role, cache with a TTL, rotate automatically. Detect leaks with secret scanning in pre-commit and CI (gitleaks, GitHub secret scanning). If a secret leaks: rotate first, then investigate usage in CloudTrail.

Q7. How would you set up WAF for a public API without breaking it?

Deploy AWS managed rule groups plus rate-based rules in Count mode, send WAF logs to S3/CloudWatch, review false positives for 1-2 weeks, add scope-down statements or exclusions for known-good patterns, then switch to Block. Add custom rules for app-specific abuse (login endpoint rate limits, geo or IP reputation). Be clear WAF is defense in depth: it doesn't fix an IDOR.

Q8. GuardDuty fires `UnauthorizedAccess:IAMUser/InstanceCredentialExfiltration.OutsideAWS`. What happened and what do you do?

The EC2 role's temporary credentials are being used from outside AWS, often after SSRF to the metadata service. Revoke sessions for the role (deny on `aws:TokenIssueTime` older than now), isolate the instance, check what the credentials did in CloudTrail, find the SSRF. Prevention: enforce IMDSv2 (`HttpTokens=required`, hop limit 1) and least privilege on the instance role. Good place to cite SSRF experience from your pentests.

CI/CD and DevSecOps tooling

The message they want: fast feedback early, block only on high-confidence, high-severity issues, and track everything else.

Q1. Design a secure pipeline. Where does each tool run?

1. IDE / pre-commit: secret scanning (gitleaks), lightweight Semgrep rules.
2. Pull request: SAST (SonarQube/Checkmarx/Semgrep) on changed code, SCA (Snyk/Dependabot), IaC scan (Checkov/tfsec), results as PR comments.
3. Build: container image scan (Trivy/Inspector/Prisma), SBOM generation (Syft, CycloneDX), image signing (cosign/Notation, AWS Signer).
4. Deploy to staging: DAST (ZAP baseline or Burp Enterprise) against the running app, authenticated where possible.
5. Deploy gate: admission control in EKS verifies signature and blocks critical CVEs; policy checks on Terraform plan.
6. Runtime: GuardDuty, Inspector continuous scanning, WAF; findings flow into Security Hub and the ticketing system.

Q2. Developers say the SAST tool is noisy and they ignore it. What do you do?

Tune before you enforce: disable low-value rules, add framework-aware config, mark false positives centrally so they don't recur. Only break the build on new, high-confidence criticals (diff-aware scanning), not the backlog. Measure false-positive rate and time-to-fix. Write custom rules for bugs you actually found in their code. Credibility comes from every blocking finding being real.

Q3. SAST vs DAST vs SCA vs IAST — strengths and blind spots?

- SAST: early, sees all code paths; weak on business logic and auth flaws, noisy on taint across frameworks.
- DAST: tests the running app as an attacker; misses unreached code, weak on complex auth and APIs without specs.
- SCA: finds known-vulnerable dependencies and license issues; needs reachability analysis to avoid alert fatigue.
- IAST: instruments the running app, low false positives; language-dependent and needs good test coverage.
- None of them find IDOR or broken business logic reliably. That's where manual review and pentesting (your strength) fit.

Q4. A critical CVE drops in a library used by 40 services. What's your process?

Use the SBOM/SCA inventory to find every affected service and version in hours, not days. Check exploitability: is the vulnerable function reachable, is the service internet-facing? Prioritize with CVSS plus EPSS and CISA KEV. Apply a WAF virtual patch if one exists while teams upgrade. Track to closure with an SLA and report status to the client.

Q5. How do you secure the pipeline itself?

The CI system is a high-value target with deploy credentials. Use OIDC federation from GitHub Actions/GitLab to AWS instead of long-lived access keys, with role trust scoped to repo and branch (`token.actions.githubusercontent.com:sub`). Pin third-party actions to commit SHAs. Protect branches, require reviews, restrict who can edit workflow files. Don't run untrusted PR code with secrets (`pull_request_target` pitfalls). Ephemeral runners. Sign artifacts and verify provenance (SLSA).

Q6. How do you run DAST against an authenticated API?

Import the OpenAPI spec into ZAP or Burp, script login to get a token (or use a long-lived test user token from Secrets Manager), configure session handling to refresh it, and scope to staging only. Add two test users in different tenants so you can check authorization, which scanners otherwise skip. Run a baseline scan per PR and a full scan nightly.

Q7. What metrics would you report to a client on their AppSec program?

Mean time to remediate by severity versus SLA, open critical/high count trend, percentage of repos with SAST/SCA/IaC scanning enabled, percentage of high-risk features threat modeled, false-positive rate, and escaped vulnerabilities found in pentest or production that tooling missed.

IaC security and guardrails

Think in layers: preventive in the pipeline (Checkov/OPA), preventive in the org (SCPs), detective at runtime (Config, Security Hub).

Q1. Preventive vs detective guardrails on AWS?

Preventive stops the action: SCPs, RCPs, permission boundaries, IaC policy checks blocking a merge, EKS admission control. Detective finds drift after the fact: AWS Config rules, Security Hub standards, GuardDuty. Use both, because people change things in the console and IaC checks never see that.

Q2. Give examples of SCPs you would apply to a client's workload OUs.

- Deny disabling or deleting CloudTrail, GuardDuty, Config, Security Hub.
- Deny actions outside approved regions (with exemptions for global services).
- Deny leaving the organization.
- Deny creating IAM users/access keys where SSO is the standard.
- Deny `s3:Put`AccountPublicAccessBlock so account-level Block Public Access can't be turned off.
- Note: SCPs don't apply to the management account, and they limit permissions but never grant them.

Q3. How do you roll out Checkov across many repos without stopping delivery?

Start in soft-fail mode and report. Pick a small baseline of critical checks to hard-fail (public S3, open security groups on 0.0.0.0/0 for 22/3389, unencrypted storage, wildcard IAM). Allow inline suppressions only with a reason and an owner, and review them. Add custom checks for client standards. Expand the hard-fail set each quarter.

Q4. Checkov/tfsec vs OPA/Conftest — when do you use which?

Checkov and tfsec ship hundreds of ready-made checks for common misconfigurations, quick to adopt. OPA with Rego (via Conftest) is a general policy engine for org-specific rules (mandatory tags, approved instance types, cost guardrails), and the same language works for Kubernetes admission (Gatekeeper). Run checks against the Terraform plan JSON, not just the HCL, so you catch values resolved from variables and modules.

Q5. Write (or describe) a Rego rule that denies public S3 buckets.

```
package terraform.s3

deny[msg] {
  r := input.resource_changes[_]
  r.type == "aws_s3_bucket_public_access_block"
  r.change.after.block_public_acls == false
  msg := sprintf("%s must block public ACLs", [r.address])
}
```

Explain that it runs against `terraform show -json plan.out`.

Q6. Terraform state contains secrets. How do you protect it?

Remote backend in S3 with SSE-KMS, versioning, Block Public Access, a bucket policy restricting access to the CI role and a break-glass role, DynamoDB locking, and access logging. Avoid putting secrets into state where possible: reference Secrets Manager ARNs

rather than values, and mark outputs `sensitive`.

Q7. How do you handle drift between Terraform and what's actually deployed?

Scheduled `terraform plan` in CI to detect drift, AWS Config to catch console changes, and an SCP or IAM policy that restricts write access in production to the pipeline role so drift can't happen in the first place.

Containers, EKS/ECS and Lambda

Your KubeArmor and Cilium work at AccuKnox is the strongest story here; use it for runtime and network policy answers.

Q1. How do you secure an EKS cluster end to end?

- Control plane: private API endpoint or restricted CIDRs, control-plane audit logs to CloudWatch, GuardDuty EKS protection.
- Identity: EKS access entries (or aws-auth) mapped to SSO roles, Kubernetes RBAC least privilege, IRSA or EKS Pod Identity so each pod gets its own IAM role instead of the node role.
- Nodes: hardened AMI (Bottlerocket), IMDSv2 with hop limit 1 so pods can't steal node credentials, managed node groups patched regularly.
- Workloads: Pod Security Standards (restricted), non-root, read-only root filesystem, dropped capabilities, no privileged or hostPath.
- Network: Kubernetes NetworkPolicies (Cilium/Calico) default-deny, security groups for pods where needed.
- Supply chain: images only from ECR, scanned and signed, admission control (Kyverno/Gatekeeper) to enforce it.
- Runtime: GuardDuty runtime monitoring, Falco or KubeArmor for process/file/network behavior enforcement.
- Secrets: Secrets Store CSI driver with Secrets Manager, envelope encryption of etcd secrets with KMS.

Q2. What did you do with KubeArmor and Cilium, and how does it apply here?

Answer with your real specifics. A strong structure: what KubeArmor enforces (process execution, file access, network at the pod level using LSMs like AppArmor/BPF-LSM), what Cilium adds (eBPF-based L3-L7 network policy, identity-aware policy, Hubble visibility), one policy you wrote and what it blocked. Then link it: "On a client's EKS cluster I'd use the same approach to move from detect-only to enforce for high-risk workloads."

Q3. ECS: task role vs task execution role?

The execution role is used by the ECS agent to pull images from ECR and fetch secrets and write logs at startup. The task role is what the application code uses at runtime to call AWS. Keep them separate and give the app only the task role permissions it needs.

Q4. Container image hardening checklist?

Minimal or distroless base image, multi-stage builds so compilers and tools don't ship, pinned base image digests, run as non-root user, no secrets baked into layers (check `docker history`), scan in CI and continuously in ECR, rebuild regularly to pick up base-image patches, sign images.

Q5. Top Lambda security risks and controls?

- Over-permissive execution role (one role shared by many functions); give each function its own scoped role.
- Event-data injection: input comes from S3 object names, SQS messages, API Gateway; validate it like any user input.
- Secrets in plain environment variables; use Secrets Manager or KMS-encrypted env vars plus the Parameters and Secrets extension.
- Vulnerable dependencies in the deployment package or layers; scan with Inspector Lambda scanning and SCA.
- Overly broad resource-based policies letting any principal invoke the function; restrict with `SourceArn` / `SourceAccount`.
- Denial of wallet: unbounded invocations; reserved concurrency, API throttling, budget alarms.
- Code signing for Lambda to allow only trusted, signed packages.

Q6. A pod in EKS is compromised. What's your response?

Isolate with a deny-all NetworkPolicy on the pod's labels (keep it running for forensics), cordon the node, capture memory/disk if needed, revoke the IAM role's sessions it could reach (IRSA role), check CloudTrail and Kubernetes audit logs for what the identity did, check for lateral movement via service account tokens. Then rebuild from a known-good image and fix the entry point.

Application and API security, auth, code review

Your home ground. Go deep, use real findings, and always end with the fix and how to prevent the class.

Q1. Pick two OWASP Top 10 items and explain how you'd prevent them across 30 teams.

Broken Access Control (A01): central authorization middleware or policy engine, deny by default, object-level checks on every data access, automated tests with two users in different tenants. Injection (A03 in 2021; check the 2025 list before the interview): parameterized queries and ORMs, output encoding by context, Semgrep rules that flag string-built SQL. Prevention at scale = secure defaults in shared libraries, not training alone.

Q2. OWASP API Security Top 10 — which issues do you see most?

BOLA (API1) and broken function-level authorization (API5), broken authentication (API2), unrestricted resource consumption (API4), and broken object property level authorization (mass assignment and excessive data exposure, API3). Give a real example: request, what you changed, what you got back, impact, fix.

Q3. Explain the OAuth 2.0 authorization code flow with PKCE and why PKCE matters.

Client generates a random code_verifier and sends its SHA-256 hash (code_challenge) in the authorization request. User authenticates at the authorization server, which redirects back with a short-lived code. Client exchanges code plus code_verifier for tokens. PKCE stops an attacker who intercepts the code (malicious app, leaked redirect) from redeeming it. Now recommended for all clients, not just mobile/SPA; implicit flow is deprecated.

Q4. OAuth 2.0 vs OIDC vs SAML?

OAuth 2.0 is delegated authorization: it issues access tokens to call APIs. OIDC adds authentication on top of OAuth, with an ID token (a JWT about the user) and a userinfo endpoint. SAML is an older XML-based federation standard for browser SSO, common in enterprise IdPs. Rule: use OIDC for login, OAuth access tokens for APIs, SAML when the enterprise IdP requires it.

Q5. Common OAuth/OIDC/SAML implementation bugs you test for?

- OAuth: open or loosely matched redirect_uri, missing state (CSRF login), token leakage via Referer or logs, missing PKCE, scope escalation, account takeover via unverified email from social login.
- JWT: `alg: none`, RS256-to-HS256 key confusion, no signature or `exp` / `aud` / `iss` validation, secrets weak enough to brute force, `kid` injection.
- SAML: signature wrapping (XSW), not validating which element is signed, comment-injection in NameID, accepting unsigned assertions, missing audience/recipient checks.

Q6. How do you do a secure code review on an unfamiliar Java/Node.js codebase?

Start with the threat model and entry points: routes/controllers, auth middleware, data access layer. Run SAST/Semgrep to get leads, then read manually along data flows from source to sink. Focus on authN/authZ checks, input handling (SQL, command, template, path, deserialization, SSRF via URL fetchers), crypto and secrets, and error handling. Check dependencies. Report with file, line, exploit scenario, severity and a concrete code fix.

Q7. Spot the bug: Node.js handler `const doc = await Doc.findById(req.params.id); res.json(doc);`

IDOR/BOLA: no check that the doc belongs to `req.user`. Fix: query with both the ID and owner/tenant (`Doc.findOne({_id: id, tenantId: req.user.tenantId})`), return 404 for not-found and not-allowed alike, and add a test.

Q8. Java: what's dangerous about deserializing untrusted data, and the fix?

`ObjectInputStream.readObject()` on attacker input can trigger gadget chains in classpath libraries leading to RCE. Fix: don't deserialize native objects from untrusted sources; use JSON with strict schemas; if unavoidable, use an allow-list filter (`ObjectInputFilter`). Same idea for Jackson polymorphic typing (avoid default typing) and .NET BinaryFormatter.

Q9. Tell us about your CVEs.

Use STAR: what Microweber is, how you found CVE-2023-2240 and CVE-2023-2014, the vulnerability class and impact, how you disclosed it, and the fix. Be precise; interviewers may look them up. End with what developers could have done to prevent it.

Incident response, detection and compliance

Show a calm sequence (contain, preserve, investigate, eradicate, recover, learn) and that you know what auditors actually ask for.

Q1. You get an alert of a spike in 401s then 200s on the login API from many IPs. What's happening and what do you do?

Likely credential stuffing that's succeeding. Contain: WAF rate-based rule on the login path, Bot Control or CAPTCHA challenge, temporary geo/IP-reputation blocks. Investigate: which accounts got 200s, from which IPs, what they did after login. Respond:

force password reset and session revocation for affected accounts, notify per client policy. Prevent: MFA, breached-password checks, per-account lockout with backoff, anomaly alerts.

Q2. How would you write detection rules for application-layer attacks?

Log the right fields first: user ID, tenant, request ID, endpoint, status, auth result, source IP. Then detect behavior: one user accessing many object IDs in sequence (BOLA enumeration), auth failures followed by success, admin endpoints called by non-admin roles, WAF block spikes. Implement as CloudWatch Logs Insights/metric filters with alarms, EventBridge rules on GuardDuty and Security Hub findings, or SIEM rules. Each rule gets a runbook.

Q3. What goes in a runbook?

Trigger and severity, owner and escalation path, first 15 minutes of triage queries, containment actions (with exact commands or console steps), evidence to preserve, communication template (internal and client), and criteria to close. Automate the safe steps with EventBridge + Lambda or SSM Automation, e.g. auto-quarantine a public S3 bucket.

Q4. How do you support a SOC 2 or ISO 27001 audit as an AppSec engineer?

Provide evidence that controls operate, not just exist: SDLC policy, threat model records, SAST/SCA/DAST scan results over the audit period, remediation tickets closed within SLA, pentest reports and retests, change management (PR reviews), access reviews, encryption and key management configs. Security Hub standards and AWS Config conformance packs help produce continuous evidence; AWS Artifact provides AWS's own reports.

Q5. PCI DSS v4.0: what matters most for an application team?

Reduce scope first (tokenize, use a payment provider, segment the cardholder data environment). Then: secure SDLC and code review for custom software (Req 6), protection of public-facing web apps with automated technical solutions such as WAF (6.4.2), inventory and integrity control of payment-page scripts (6.4.3), strong authentication with MFA for CDE access (Req 8), logging (Req 10), and annual pentests plus segmentation testing (Req 11).

Q6. HIPAA or FedRAMP in one line each?

HIPAA: protect PHI with access control, audit logging, encryption and a BAA with AWS, using only HIPAA-eligible services. FedRAMP: US federal cloud authorization based on NIST 800-53 controls; workloads usually run in GovCloud with strict boundary and

continuous monitoring requirements.

Q7. How do you coordinate a third-party pentest for a client?

Agree scope, rules of engagement and test window; get written authorization; prepare test accounts and environments; notify the SOC and AWS-relevant teams (AWS no longer needs pre-approval for most services, but check the policy). During the test, be the point of contact for blockers. After: triage findings, remove false positives, assign owners and SLAs, verify fixes in a retest, and report status to stakeholders.

Scenario and behavioral questions

They are testing judgment and communication with clients. Answer in STAR (situation, task, action, result) with a number in the result; prepare a real story for each before the interview.

Q1. A dev lead says your critical finding will delay a release and wants to ship anyway. What do you do?

Confirm exploitability and impact with evidence, not severity labels. Offer options: a quick mitigation (feature flag off, WAF rule, tighter IAM) that lets them ship safely, with a dated fix ticket. If the risk is still unacceptable, escalate to the risk owner with a clear write-up so the business makes an informed decision and it's documented. You don't own the decision; you own making the risk clear.

Q2. Explain a technical vulnerability to a non-technical executive.

Practice live with an IDOR: "Any logged-in customer could view any other customer's invoices by changing a number in the address. That's roughly N customers' data exposed, it's a reportable breach under our obligations, and the fix is two days of work." Impact, likelihood, cost to fix; no jargon.

Q3. You join a client program with no AppSec in place. What do you do in the first 90 days?

1. Days 1-30: inventory apps, repos, AWS accounts and data; quick wins (Security Hub + GuardDuty on, secret scanning, block public S3); meet team leads.
2. Days 31-60: SCA and IaC scanning in pipelines in report-only mode, threat model the top 3 riskiest apps, define severity SLAs.
3. Days 61-90: enforce a small set of blocking checks, launch a security champions group, first metrics report to the client.

Q4. Tell me about a time you disagreed with a developer and how it resolved.

Pick a real one. The good shape: you listened, reproduced the issue together, found a fix that fit their architecture, and the relationship improved.

Q5. How do you balance security rigor with delivery speed?

Risk-based: block only what is exploitable and serious, automate the rest into tickets, give secure defaults and paved roads so the safe way is the easy way, and do design reviews early when changes are cheap. Measure that security isn't the bottleneck (review turnaround time).

Q6. How have you mentored engineers or built security culture?

Security champions, short demos of real exploits from their own codebase (more convincing than slides), secure coding checklists per stack, office hours, celebrating fixes. Give one concrete example with a result.

Q7. How do you use AI in your security work? (Likely, given Tenarai's focus)

This is your differentiator. Describe concrete workflows: Claude-assisted traffic analysis through Burp MCP, multi-agent code audits, triage and report drafting. Then the guardrails: human verification of every finding, no client data sent to tools without approval, and awareness that LLM output can be wrong. Mention you can also secure AI systems: prompt injection, agent tool permissions, RAG data isolation.

Q8. Why Tenarai, and why leave pure pentesting?

You want to prevent vulnerabilities at scale, not only find them. Tenarai builds AI and cloud systems for large enterprises, so the security problems are new (agentic AI) and the impact is broad. Your offensive background makes your guardrails realistic.

Questions to ask them and final checklist

Ask two or three of these; they show you're thinking like a consultant, not just a tester.

Questions to ask

- Is this role embedded with one client long-term or spread across several engagements?
- Do you have a standard DevSecOps toolchain you bring to clients, or do you adopt each client's tools?
- Which CNAPP (Wiz, Prisma Cloud, CrowdStrike) do clients most often use?

- How does security fit into the Hybrid Agentic PODs: are AI agents generating code that goes through the same SAST/SCA gates?
- Who owns security of Tenarai-IGNIS, and would this role touch LLM/agent security?
- What does success look like in the first 6 months?

Final prep checklist

- [ ] Rehearse the 60-second intro out loud
- [ ] Write 2 real AWS security stories in STAR form, with numbers
- [ ] Prepare the KubeArmor/Cilium story with one specific policy and outcome
- [ ] Refresh both CVE details (class, root cause, fix)
- [ ] Run Checkov and Trivy on a sample Terraform repo and container image, so you've used them hands-on
- [ ] Write one Semgrep rule and one Rego policy from scratch
- [ ] Review the latest OWASP Top 10 and OWASP Top 10 for LLM Applications
- [ ] Be able to draw the AWS architecture and threat model in Q1 of threat modeling on a whiteboard in 5 minutes
- [ ] Decide what you'll say about AWS Security Specialty (plan and date)